Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

Python
Learn
Tutorials
Deep Agents
Data Analysis
Deep Research
Content Builder
RAG
LangChain
Deep Agent
Semantic search

Retrieval Augmented Generation (RAG) with Deep Agents

Copy page

On this page
RAG patterns
Why retrieval matters
What you will build
Prerequisites
Setup
Index LangChain documentation
Load documents
Split documents
Select an embeddings model
Store chunks and embeddings in VectorStore
Build the agent

One of the most powerful LLM-based applications are sophisticated question-answering (Q&A) chatbots which augment LLMs by providing it with inference-time access to a set of data. This might be private data, recent data, or data that is not part of the training data the LLM is trained on. These applications use a technique known as Retrieval Augmented Generation, or RAG.

Deep Agents gives you primitives for RAG: custom retrieval tools, a filesystem backend, subagents, skills, and grading rubrics. You can combine them in different ways depending on your corpus size, latency requirements, and how strictly answers must be grounded in source data.

This guide introduces several RAG patterns and walks through one end-to-end example: a documentation Q&A agent that indexes a subset of docs.langchain.com, retrieves relevant chunks at query time, offloads them to the filesystem, and delegates analysis to subagents so the orchestrator context stays clean.

RAG patterns

Deep Agents allows you to orchestrate retrieval, analysis, and synthesis in several ways:

- **Skills-guided retrieval**: The user asks a question. The agent loads a relevant skill that describes how to search your corpus (which index to use, query formulation, citation format). The agent calls your retrieval tool following that guidance, then synthesizes an answer.
- **Rubric-checked grounding**: The user asks a question. The agent retrieves evidence and drafts an answer. A grader sub-agent, configured with `RubricMiddleware`, evaluates whether the response is grounded in the retrieved source material. The agent revises until the rubric passes or an iteration cap is reached.
- **Todo-driven investigation**: The user asks a question. If you opt into task planning, the agent uses the planning tool to create a todo list of documentation pages or search queries to investigate. It retrieves results for each item, then synthesizes a response from the collected evidence.
- **Retrieve, offload, and delegate**: The user asks a question. The agent retrieves matching chunks and writes them to the filesystem backend rather than keeping full text in the orchestrator context. Subagents read, search, and summarize individual files in parallel. For large documents, the agent can paginate through files with built-in search tools or run a code interpreter to produce tables, timelines, or visuals from source data.

Grading rubrics require `deepagents>=0.6.5` and are currently in beta.

This tutorial implements the **retrieve, offload, and delegate** pattern. The same primitives appear in the other patterns: skills often wrap retrieval workflows, rubrics can grade any of these flows, and opt-in todo planning helps break complex questions into focused searches.

Why retrieval matters

A language model on its own does not have access to your documentation. Ask it about a specific API that changed recently, and it answers from training data: often plausible, sometimes wrong, and never grounded in your source of truth.

Even when documentation is available, you generally cannot just fit it all into the context window. You therefore must select only the passages relevant to a given question, which in itself is a non-trivial task.

This tutorial uses one question throughout:

> How do I stream intermediate tool results from a subagent?

Pass that question to a Deep Agent with no custom tools and no access to the documentation corpus, to see what the model comes up with:

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama

```python
from deepagents import create_deep_agent
from langchain.messages import HumanMessage

EXAMPLE_QUERY = "How do I stream intermediate tool results from a subager

baseline_agent = create_deep_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[],
    system_prompt=(
        "You are a helpful LangChain documentation assistant. "
        "Answer questions about LangChain APIs and patterns."
    ),
)

result = baseline_agent.invoke(
    {"messages": [HumanMessage(content=EXAMPLE_QUERY)]}
)

print(result["messages"][-1].text)
```

View example trace
Open a public LangSmith run for this example.

Without retrieval, the agent cannot look up current LangChain documentation. Responses tend to be generic, may omit guidance such as subagent streaming, or include outdated information.

The example in this tutorial indexes LangChain documentation, retrieves evidence with a vector search tool, analyzes each chunk in parallel subagents, and answers a question with citations to the docs.

What you will build

1. **Index**: Load the LangChain documentation into a vector store.
2. **Search**: Build a custom tool that runs vector similarity search and writes each retrieved chunk to the agent filesystem.
3. **Analyze**: Delegate file analysis to a subagent that reads the file and returns a focused summary.
4. **Synthesize**: Use the main agent to get the final answer from subagent reports.

Prerequisites

API keys for:

- A chat model integration for the agent
- OpenAI (or another embeddings integration) for indexing

Setup

1. Create project directory

```bash
mkdir docs-rag-agent
cd docs-rag-agent
```

2. Install dependencies

pip uv

```bash
pip install deepagents "langchain[openai]" langchain-text-splitters requests numpy
```

3. Set API keys

```bash
export OPENAI_API_KEY="your_openai_api_key"
export ANTHROPIC_API_KEY="your_anthropic_api_key"   # If using Clauc
export GOOGLE_API_KEY="your_google_api_key"         # If using Gemir
```

For any other provider, please see the respective chat model documentation.

4. Set up LangSmith

RAG applications run retrieval and generation in sequence. When you run the examples in this tutorial, LangSmith logs a trace for each query so you can inspect retrieval, tool calls, and model responses. After you sign up for LangSmith, set your environment variables to start logging traces:

```bash
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="..."
```

Or, set them in Python:

```python
import getpass
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

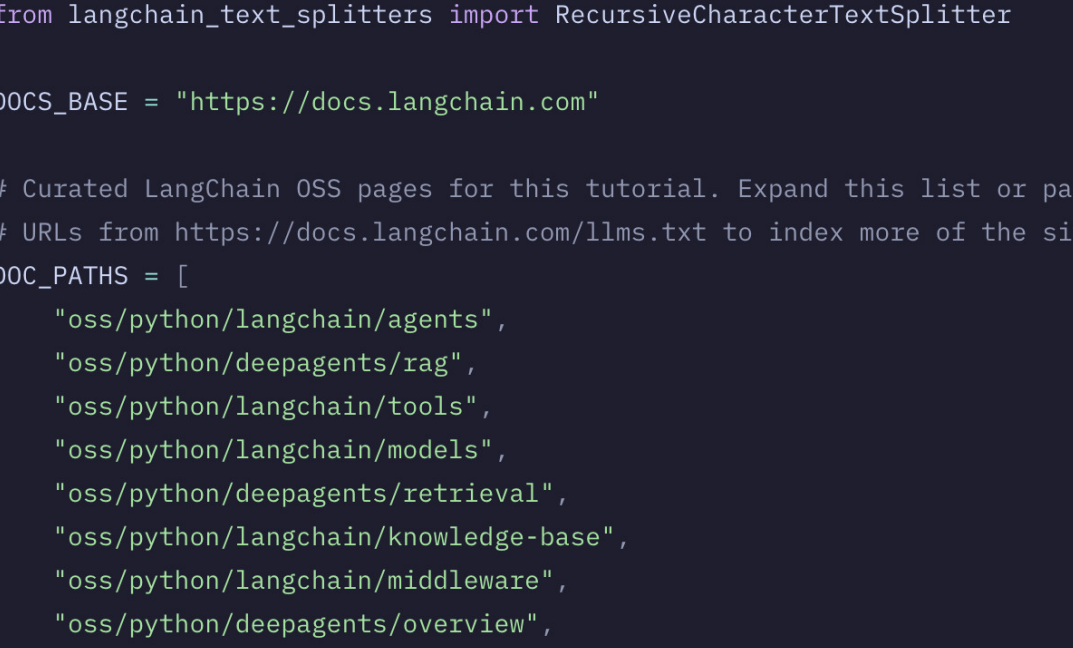
If you are building a production agent, we also recommend you set up LangSmith Engine which monitors your traces, detects issues, and proposes fixes.

Index LangChain documentation

In the indexing step, you'll take the source content and convert *chunks* of it into numerical representations. This numerical representation captures the semantic meaning of the chunk. Storing a mapping of these numerical representations and the document chunks in a `VectorStore` allows you to efficiently retrieve relevant content when a user sends a query based on its own numerical representation.

Indexing commonly works in four steps:

1. **Load**: Load your data sources into `Document` objects.
2. **Split**: Use text splitters to break large `Document`s into smaller chunks. This is useful both for indexing data and passing it to a model, as large chunks are harder to search over and either do not fit in a model's finite context window or use more tokens than necessary.
3. **Embed**: Embeddings models convert each chunk into a numeric vector that captures its meaning, enabling similarity search over your content.
4. **Store**: Use a VectorStore to index chunks and their embeddings for retrieval.



In the indexing step, fetch documentation pages, split them into chunks, embed the chunks, and store them in a `VectorStore`. The agent searches this index at runtime; it does not re-fetch the full site on every question.

LangChain publishes markdown at `https://docs.langchain.com/{path}.md`. This tutorial indexes a curated list of open source documentation paths. You can expand `DOC_PATHS` or parse URLs from llms.txt to cover more pages.

Create `agent.py`:

```python
import requests
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

DOCS_BASE = "https://docs.langchain.com"

# Curated LangChain OSS pages for this tutorial. Expand this list or pars
# URLs from https://docs.langchain.com/llms.txt to index more of the site
DOC_PATHS = [
    "oss/python/langchain/agents",
    "oss/python/deepagents/rag",
    "oss/python/langchain/tools",
    "oss/python/langchain/models",
    "oss/python/deepagents/retrieval",
    "oss/python/langchain/knowledge-base",
    "oss/python/langchain/middleware",
    "oss/python/deepagents/overview",
    "oss/python/deepagents/subagents",
    "oss/python/deepagents/streaming",
    "oss/python/deepagents/frontend/subagent-streaming",
    "oss/python/deepagents/backends",
    "oss/python/langgraph/overview",
    "oss/python/langgraph/quickstart",
]
```

For a more detailed tutorial on indexing, vector stores, and retrieval, see Semantic search.

Load documents

Start by loading LangChain documentation pages into a list of Document objects.

Use `requests` to fetch each page as markdown from `https://docs.langchain.com/{path}.md`. The curated `DOC_PATHS` list selects which pages to index.

```python
def load_langchain_docs(doc_paths: list[str] | None = None) -> lis
    """Fetch LangChain documentation pages as Documents."""
    paths = doc_paths or DOC_PATHS
    docs: list[Document] = []
    for path in paths:
        url = f"{DOCS_BASE}/{path}.md"
        try:
            response = requests.get(url, timeout=20)
            response.raise_for_status()
        except requests.RequestException:
            continue
        source = f"{DOCS_BASE}/{path}"
        docs.append(
            Document(page_content=response.text, metadata={"source": sour
        )
    return docs


docs = load_langchain_docs()
print(f"Loaded {len(docs)} documentation pages.")
```

If you run this code it prints:

```
Loaded 14 documentation pages.
```

You can also review the page content itself:

```python
total_chars = sum(len(doc.page_content) for doc in docs)
print(f"Total characters: {total_chars}")
print(docs[0].page_content[:500])
```

```
Total characters: 589579
> ## Documentation Index
> Fetch the complete documentation index at: https://docs.langchain.com/l
> Use this file to discover all available pages before exploring further.

# Build a RAG agent with LangChain
```

Split documents

The loaded documentation is long with over 100k tokens total, which makes it too large to fit into the context window of many models. Even for those models that could fit the full corpus in their context window, models can struggle to find information in very long inputs. Using the context window for large amounts of content is also not token efficient.

For ease of use, split the `Document` objects into chunks. These chunks will be used for embedding and vector storage in the next steps.

Use the `RecursiveCharacterTextSplitter` to recursively split the documents using common separators like new lines, until each chunk is the appropriate size. `RecursiveCharacterTextSplitter` is the recommended `TextSplitter` for generic text use cases.

```python
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, ch
all_splits = text_splitter.split_documents(docs)
print(f"Split documentation into {len(all_splits)} chunks.")
```

```
Split documentation into 782 chunks.
```

If you want to learn more about text splitters, check out the `TextSplitter` interface and text splitter integrations.

Select an embeddings model

An embedding is a numeric vector that captures the meaning of each documentation chunk. An Embeddings model converts those chunks into vectors so that similar meanings land close together in vector space, enabling you to retrieve relevant sections when a user asks a question.

You can choose from many different embedding integrations which all use the same Interface:

OpenAI Azure Google Gemini Gemini Enterprise Agent Platform AWS HuggingFa

```bash
pip install -U "langchain-openai"
```

```python
import getpass
import os

if not os.environ.get("OPENAI_API_KEY"):
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter API key for Ope

from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
```

Store chunks and embeddings in VectorStore

A `VectorStore` persists document chunks and their embeddings, enabling similarity search to retrieve relevant sections when a user asks a question. You can choose from many different vector store integrations which all use the same Interface. Use the embeddings model that you selected in the previous step to configure your `VectorStore`:

In-memory Amazon OpenSearch AstraDB Chroma Milvus MongoDB PGVecto

```bash
pip install -U "langchain-core"
```

```python
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore(embeddings)
```

Then, embed and store all document splits using the `vector_store` you initialized above:

```python
vector_store.add_documents(documents=all_splits)
print(f"Indexed {len(all_splits)} chunks.")
```

When run, this outputs:

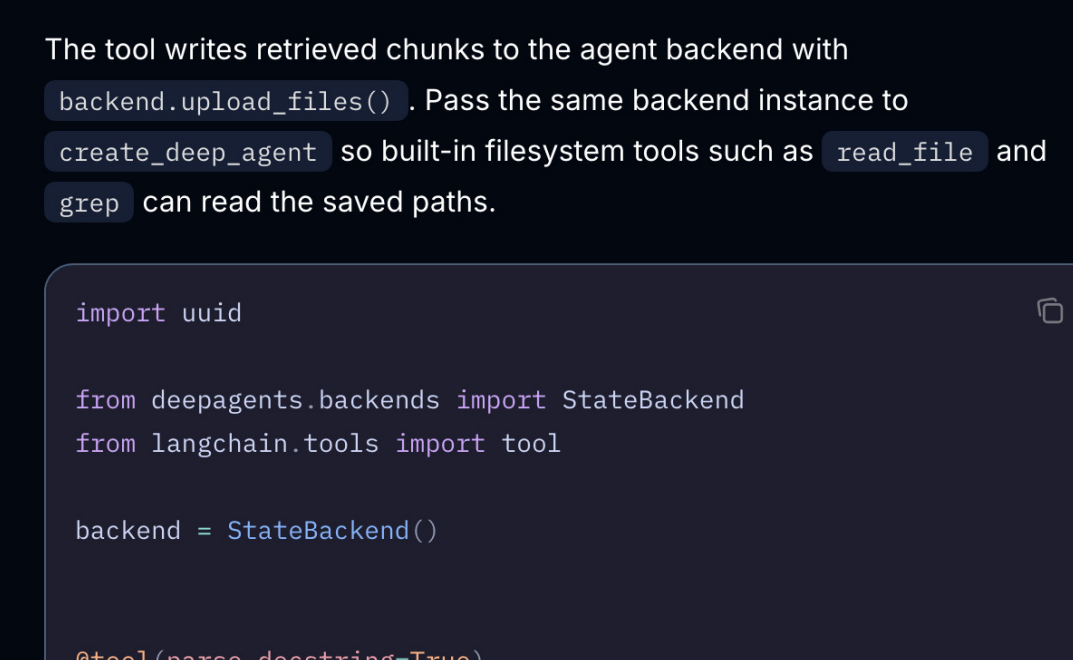
```
Indexed 782 chunks.
```

Indexing runs once at startup in this tutorial. In production, persist the vector store to disk or a hosted vector database and refresh it on a schedule when documentation changes.

This completes the **Indexing** portion of the tutorial. You now have a queryable vector store containing chunked LangChain documentation.

The next step is to build a Deep Agent that searches this index at run time, offloads retrieved chunks to the filesystem, and delegates analysis to subagents. See Build the agent. To think of it in RAG terms:

1. **Retrieve**: Given a user input, relevant splits are retrieved from storage using a Retriever.
2. **Generate**: A model produces an answer using a prompt that includes both the question and the retrieved data.



Build the agent

Add this code to `agent.py`:

1. Add the search tool

The `search_documentation` tool runs similarity search against the indexed corpus, then writes each retrieved chunk to the agent filesystem under `/retrieved/{batch_id}/`. It returns file paths so the orchestrator can delegate analysis without loading full chunk text into its context.

The tool writes retrieved chunks to the agent backend with `backend.upload_files()`. Pass the same backend instance to `create_deep_agent` so built-in filesystem tools such as `read_file` and `grep` can read the saved paths.

```python
import uuid

from deepagents.backends import StateBackend
from langchain.tools import tool

backend = StateBackend()


@tool(parse_docstring=True)
def search_documentation(query: str) -> str:
    """Search LangChain documentation and save matching chunks to th

    Args:
        query: Natural language search query.

    Returns:
        File paths where retrieved chunks were saved under /retrieve
    """
    retrieved_docs = vector_store.similarity_search(query, k=4)
    batch_id = uuid.uuid4().hex[:8]
    uploads: list[tuple[str, bytes]] = []
    saved_paths: list[str] = []

    for index, doc in enumerate(retrieved_docs, start=1):
        path = f"/retrieved/{batch_id}/chunk_{index}.md"
        content = (
            f"# Source: {doc.metadata.get('source', 'unknown')}\n\n'
            f"{doc.page_content}"
        )
        uploads.append((path, content.encode("utf-8")))
        saved_paths.append(path)

    backend.upload_files(uploads)
    return (
        f"Saved {len(saved_paths)} documentation chunks:\n"
        + "\n".join(saved_paths)
    )
```

2. Add prompts

Add the orchestrator workflow and subagent prompt templates to `agent.py`:

```
RAG_WORKFLOW_INSTRUCTIONS = """# Documentation Q&A workflow

Answer questions about LangChain using the indexed documentation
corpus.

1. **Plan**: Break complex questions into focused search queries.
2. **Search**: Call search_documentation with a query. The tool
```

⋯ See all 13 lines

```
CHUNK_ANALYST_INSTRUCTIONS = """You analyze retrieved LangChain
documentation chunks stored as markdown files.

Your task description includes the user's question and one file
path under /retrieved/.

Use read_file to read the assigned chunk. Extract facts that help
```

⋯ See all 10 lines

```
SUBAGENT_DELEGATION_INSTRUCTIONS = """# Subagent coordination

Your role is to coordinate chunk analysis by delegating to the
chunk-analyst subagent.

## Delegation strategy
```

⋯ See all 16 lines

3. Create the agent

Add model initialization and agent creation to `agent.py`:

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama

```python
from deepagents import create_deep_agent
from langchain.chat_models import init_chat_model

max_concurrent_analysts = 3

INSTRUCTIONS = (
    RAG_WORKFLOW_INSTRUCTIONS
    + "\n\n"
    + "=" * 80
    + "\n\n"
    + SUBAGENT_DELEGATION_INSTRUCTIONS.format(
        max_concurrent_analysts=max_concurrent_analysts,
    )
)

chunk_analyst_subagent = {
    "name": "chunk-analyst",
    "description": (
        "Analyze one retrieved documentation chunk file. "
        "Pass the user question and a single file path under /retri
    ),
    "system_prompt": CHUNK_ANALYST_INSTRUCTIONS,
}

model = init_chat_model(model="google_genai:gemini-3.6-flash")

agent = create_deep_agent(
    model=model,
    tools=[search_documentation],
    backend=backend,
    system_prompt=INSTRUCTIONS,
    subagents=[chunk_analyst_subagent],
)
```

The main agent keeps the `search_documentation` tool. The `chunk-analyst` subagent uses built-in filesystem tools to read chunk files but does not search the vector store directly.

Run the agent

Run the RAG agent with the example query:

```python
from langchain.messages import HumanMessage

EXAMPLE_QUERY = "How do I stream intermediate tool results from a subagen

if __name__ == "__main__":
    result = agent.invoke(
        {"messages": [HumanMessage(content=EXAMPLE_QUERY)]}
    )

    for msg in result.get("messages", []):
        if msg.text:
            print(msg.text)
```

When the agent runs, it:

1. Calls `search_documentation` with a query about subagent streaming.
2. Receives file paths such as `/retrieved/a1b2c3d4/chunk_1.md`.
3. Launches one or more `task()` calls to `chunk-analyst`, each scoped to a single chunk file.
4. Synthesizes a final answer with links to the relevant documentation pages.

If you enabled LangSmith in Setup, open LangSmith and inspect the trace to see search calls, filesystem writes, subagent delegations, and the final response.

Security considerations

RAG applications are susceptible to **indirect prompt injection**. Retrieved documentation may contain text that resembles instructions. Because retrieved chunks share the context window with your system prompt, models may follow instructions embedded in documentation rather than your intended prompt.

No prompt or delimiter strategy fully prevents indirect prompt injection. The orchestrator and subagent prompts in this tutorial ask the model to treat retrieved content as data only, and the search tool prefixes chunks with a `# Source:` header so analysts can distinguish metadata from body content. These patterns can help in some cases, but they do not provide reliable protection.

Validate agent outputs before surfacing them to users. Check that answers cite expected documentation paths and that claims match the retrieved source material.

For more on this topic, see research on prompt injection.

Full code

The following is the complete script for the agent using one set of example models. For other models please see the step-by-step approach to see what changes:

Save as `agent.py` and run with `python agent.py`:

```python
import uuid

import requests
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage
from langchain.tools import tool
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
```